

QWeave Complete Research and Publication Workflow

Overall operating procedure for a proof-backed quantum compiler project

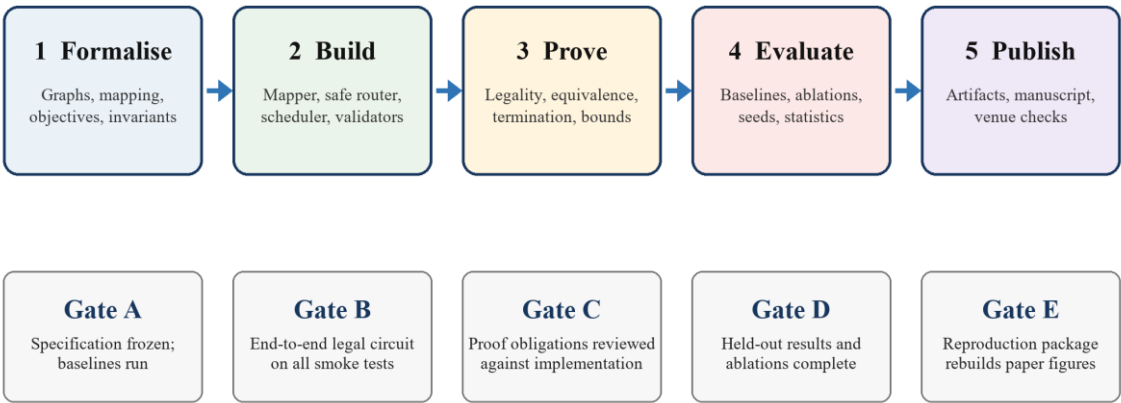
Team: Shrivardhini N | Atharva Sheersh Pandey | Diptesh Das | Haridasu Sreedhar
Guide: Prof. Bhuvaneswari M | Planning horizon: 21 days | Version: 1.0

Purpose and Outcome

This document is the shared operating manual for taking QWeave from a research idea to a reproducible compiler prototype and a submission-ready journal manuscript in 21 days. It describes the overall workflow rather than assigning work to individuals. The project succeeds only when the implementation, formal arguments, tests, experiments, and manuscript tell the same story.

The central contribution should be framed as a proof-backed compiler pipeline that combines interaction-aware initial placement, a topology-aware masked GNN-PPO router, and depth-aware scheduling. The mathematical claims establish safety and correctness; the experimental claims establish performance. A learned router cannot honestly be claimed to be globally optimal.

QWeave End-to-End Research Workflow



Every gate is evidence-based: code + test + result/proof + reproducibility record

Figure 1. The five-stage workflow and the evidence gate required after each stage.

1 Definition of Success

Dimension	Minimum acceptable result	Journal-strength target
Compiler	End-to-end legal output on all smoke tests	Modular Qiskit passes with deterministic fallback and documented interfaces
Mathematics	Correctness, legality, scheduler feasibility, fallback termination	Proofs linked to executable invariants and counterexample tests
Performance	Competitive with a reproducible SABRE baseline	Statistically significant depth reduction on held-out circuits and topologies

Dimension	Minimum acceptable result	Journal-strength target
Evidence	At least five seeds and fixed test split	Ablations, confidence intervals, effect sizes, timeout reporting, complete artifact bundle
Publication	Complete manuscript and references	Venue-aligned paper, anonymised package if required, data/code availability statement

2 Scope Freeze Before Coding

Freeze the research question by the end of Day 1. Avoid adding noise-aware placement, pulse scheduling, multi-programming, error mitigation, or real-hardware execution unless the core pipeline is already stable. Those are extension studies, not prerequisites.

- Primary question: does the complete pipeline reduce scheduled two-qubit depth relative to matched routing baselines on held-out circuits?
- Secondary questions: which stage contributes the improvement; how well does the router generalise to unseen coupling graphs; what runtime is paid for the depth gain?
- Hard constraints: semantic equivalence, injective logical-to-physical mapping, connectivity-valid two-qubit gates, dependency preservation, and resource-safe scheduling.
- Primary metric: scheduled two-qubit depth after a common native decomposition. Secondary metrics: total depth, SWAP count, two-qubit count, duration, estimated success probability, compiler time, and failures/timeouts.
- Baselines: identity/random layout, the project heuristic without learning, Qiskit SABRE with fixed seeds, and an exact or CP-SAT oracle on small instances where tractable.

2.1 The Publishable Claim

A defensible claim is: QWeave provides a safe, modular routing architecture in which action masking and a deterministic fallback guarantee legal progress, while a depth-aligned learned policy improves median scheduled depth on the stated evaluation distribution. The words 'guarantee' apply only to the proved safety properties. The words 'improves' and 'generalises' require held-out experimental evidence.

3 End-to-End Engineering Workflow

3.1 Repository and environment

Create a src-layout Python package; pin the environment after a one-day compatibility spike; enable CI for lint, unit tests, and a small end-to-end compilation.

Keep configs, raw results, processed tables, figures, and manuscript sources in separate directories. Every figure must be regenerable from a committed script and immutable result file.

3.2 Circuit intake and normalisation

Accept OpenQASM 2/3 or Qiskit QuantumCircuit input. Decompose consistently to a declared logical gate set, remove measurements during optimisation if appropriate, retain classical dependencies, and construct a DAG.

Reject circuits wider than the device. Record logical width, original depth, two-qubit count, and circuit family before any optimisation.

3.3 Hardware-model adapter

Represent the device with Qiskit's Target and a canonical internal graph. Store directed connectivity, supported instructions, duration, and optional error. Use the same snapshot for every method in a comparison.

Unit-test directionality, disconnected graphs, isolated qubits, missing durations, and symmetric/undirected simplifications.

3.4 Heuristic initial mapping

Build a weighted logical-interaction graph using frequency, temporal proximity, and criticality. Seed physical placement from device centrality and interaction pressure; improve it with strictly accepted pair swaps.

Return the mapping, objective value, iteration trace, and deterministic seed. Verify injectivity after every accepted move.

3.5 Routing environment

The state contains the current mapping, executable frontier, look-ahead gates, device node/edge attributes, and occupancy. Candidate actions are coupling edges representing SWAPs plus EXECUTE when a frontier gate is legal.

An action mask removes illegal actions before sampling. A progress watchdog hands control to a deterministic shortest-path fallback if the policy cycles or stalls.

3.6 GNN-PPO policy

Encode the hardware graph with message passing; pool circuit-frontier context; score device-edge actions; train actor and critic with PPO. Begin with small circuits and regular graphs, then widen the curriculum.

Log every reward component, entropy, KL estimate, clipping fraction, episode length, invalid-action count, fallback rate, and evaluation depth.

3.7 Depth-aware scheduling

Construct the routed dependency DAG, compute reverse critical-path ranks with gate durations, and schedule ready operations at the earliest resource-feasible time. Include single-qubit gates and any direction-correction gates.

Produce both an ordered circuit and a timing table. Recheck dependencies, qubit overlap, target timing constraints, and final makespan.

3.8 Validation

Run structural legality checks on every output. For small circuits, compare statevectors or unitaries up to global phase and the declared final permutation. Use metamorphic tests: compile equivalent circuit rewrites and compare results.

No benchmark result is admitted unless the output passes validation. Failure counts remain visible rather than being silently dropped.

4 Mathematical Workflow

Write the formal model before writing the proof. Each implementation object must have an explicit mathematical counterpart: circuit DAG, hardware graph, mapping, action, scheduled operation, and final permutation.

1. State the assumptions: finite circuit; connected hardware component containing enough physical qubits; finite positive gate durations; deterministic legality guard; and a fallback that routes along shortest paths.
2. Define invariants and attach each one to an assertion in code. Examples: mapping injectivity, action legality, monotone frontier progress under fallback, dependency preservation, and non-overlapping resource intervals.
3. Prove local lemmas first: a SWAP preserves injectivity; a masked policy has zero probability on illegal actions; an accepted local-search move reduces the mapping objective.
4. Compose the lemmas into end-to-end theorems: hardware legality, semantic equivalence up to final permutation, schedule feasibility, and termination with fallback.
5. Treat performance separately. Report lower bounds and objective alignment, but use experiments—not proof language—to establish reduced depth.
6. Review every theorem against the implementation. If the code violates an assumption, either fix the code or weaken the theorem.

Proof obligation	Evidence in code	Counterexample test
Mapping remains injective	Permutation update and assertion	Repeated physical qubit in a mapping
Only legal two-qubit gates execute	Target edge guard and action mask	Directionally unsupported edge
Fallback terminates	Shortest-path distance decreases	Disconnected device or stalled frontier
Scheduling is feasible	Precedence and interval checks	Two gates overlap on one qubit
Output is equivalent	Final-layout-aware unitary/statevector check	Omitted SWAP permutation

5 Experimental Protocol

- Dataset: choose a balanced subset from MQT Bench and QASMBench plus controlled synthetic interaction graphs. Stratify by width, depth, density, and circuit family [R6, R7].
- Split before tuning: circuit-family holdout and topology holdout. Do not let variants of the same circuit leak across training and test sets.
- Run policy and stochastic baselines with at least five fixed seeds; ten is preferable if runtime permits. Store per-circuit paired results.
- Use identical decomposition, target, timeout, and machine. Report environment and commit hashes.
- Analyse medians, interquartile ranges, bootstrap 95% confidence intervals, paired Wilcoxon signed-rank tests, effect sizes, and Holm correction for multiple comparisons.
- Report all timeouts and failures. A method that fails cannot be compared only on the easy subset it completes.

5.1 Mandatory Ablations

Ablation	Question answered	Expected interpretation
No learned router	Does PPO add value beyond the heuristic/fallback?	Isolates the learned decision rule
MLP instead of GNN	Does topology-aware message passing matter?	Tests the graph inductive bias
No depth-aware rescheduling	Does the scheduler create additional improvement?	Separates routing from realised depth

Ablation	Question answered	Expected interpretation
No initial-map refinement	Does monotone local search reduce routing pressure?	Tests the initial placement stage
No potential shaping	Does shaped feedback improve learning without changing the intended optimum?	Tests reward design and training stability

6 Three-Week Stage Gates

Deadline	Gate	Required evidence
Day 3	Specification gate	Frozen objective, splits, target graphs, baseline harness, theorem assumptions
Day 7	Deterministic core gate	Heuristic mapper, fallback router, scheduler, validation on smoke suite
Day 11	Learning gate	Gymnasium environment passes checker; masked GNN policy runs end to end
Day 14	Training freeze	Chosen hyperparameters frozen; held-out set remains untouched
Day 17	Results freeze	Baselines, ablations, seeds, tables, plots, failure report complete
Day 19	Manuscript freeze	Proofs, methods, results, limitations, reproducibility statement complete
Day 21	Submission gate	Repository tagged; artifact rebuild passes; venue checklist and files complete

7 Publication Workflow and Venue Strategy

IEEE Transactions on Quantum Engineering is the most natural first-fit engineering venue because its stated scope includes quantum computation and software [R12]. Quantum Science and Technology is a stretch target: its scope includes quantum software and machine intelligence, but it explicitly describes itself as highly selective and expects broad, lasting impact [R13]. Journal quartiles change by database, category, and year; verify the current Scopus/SJR or JCR quartile immediately before submission and do not copy an old quartile claim into the cover letter.

A submission in three weeks is plausible only if the result is already stable by Day 17. Acceptance is a later peer-review outcome and cannot be scheduled. If the full learned system is not convincingly better, submit a narrower proof-backed compiler methodology only after strengthening its novelty; do not conceal a negative result.

- Choose one target journal and download its current template and author checklist.
- Write the abstract around problem, method, formal guarantees, evaluation scale, quantitative result, and limitation.
- Create an anonymous manuscript if double-anonymous review is selected; remove author metadata from PDF and repository links.
- Prepare a cover letter stating novelty, strongest quantitative result, proof contribution, and why the paper fits the journal.
- Archive code, configurations, circuit lists, raw results, processed tables, and a one-command reproduction path.
- Perform plagiarism, citation, authorship, conflict-of-interest, and AI-use policy checks before submission.

8 Definition of Done

- A clean environment can run the smoke suite and rebuild the main tables/figures from committed files.
- Every reported circuit result is linked to a configuration, seed, input hash, output validation status, and software commit.
- All proofs state assumptions and match code assertions; no theorem claims global depth optimality of PPO.
- All baselines use the same target, gate decomposition, seeds, timeout, and metric implementation.
- The repository, paper, supplementary material, authorship order, and venue requirements have been reviewed by all four authors and the guide.

References and Primary Sources

- [R1] G. Li, Y. Ding, and Y. Xie, 'Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices,' ASPLOS, 2019. doi:10.1145/3297858.3304023.
- [R2] A. Cowtan et al., 'On the Qubit Routing Problem,' arXiv:1902.08091, 2019.
- [R3] J. Schulman et al., 'Proximal Policy Optimization Algorithms,' arXiv:1707.06347, 2017.
- [R4] J. Gilmer et al., 'Neural Message Passing for Quantum Chemistry,' ICML, 2017.
- [R5] A. Y. Ng, D. Harada, and S. Russell, 'Policy Invariance under Reward Transformations,' ICML, 1999.
- [R6] N. Quetschlich, L. Burgholzer, and R. Wille, 'MQT Bench,' Quantum, vol. 7, 1062, 2023. doi:10.22331/q-2023-07-20-1062.
- [R7] A. Li et al., 'QASMBench: A Low-level QASM Benchmark Suite for NISQ Evaluation and Simulation,' arXiv:2005.13018, 2020.
- [R8] IBM Quantum Documentation, 'Transpiler stages' and 'Transpiler API,' <https://quantum.cloud.ibm.com/docs/en/guides/transpiler-stages> (accessed 4 September 2026).
- [R9] PyTorch Geometric Documentation, 'Creating Message Passing Networks,' https://pytorch-geometric.readthedocs.io/en/latest/tutorial/create_gnn.html (accessed 4 September 2026).
- [R10] Gymnasium Documentation, 'Make your own custom environment,' https://gymnasium.farama.org/main/tutorials/environment_creation/ (accessed 4 September 2026).
- [R11] Google OR-Tools Documentation, 'Constraint Optimization and CP-SAT,' <https://developers.google.com/optimization/cp/> (accessed 4 September 2026).
- [R12] IEEE Transactions on Quantum Engineering, 'Scope and Submission Process,' <https://tqe.ieee.org/submission-process/> (accessed 4 September 2026).
- [R13] IOP Publishing, 'About Quantum Science and Technology,' <https://publishingsupport.iopscience.iop.org/journals/quantum-science-technology/about-quantum-science-technology/> (accessed 4 September 2026).